

Practical Week-8

Task 1: Develop a program using malloc(), calloc(), realloc(), and free(). Monitor memory allocation behavior and identify memory leaks using Valgrind.

```
GNU nano 8.7.1 memory.c *
#include <stdio.h>
#include <stdlib.h>

int main()
{
    // malloc
    int *a = malloc(5 * sizeof(int));

    if (a == NULL)
    {
        printf("malloc failed\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        a[i] = i + 1;

    printf("malloc: allocated 5 integers\n");

    // calloc
    int *b = calloc(5, sizeof(int));

    if (b == NULL)
    {
        printf("calloc failed\n");
        free(a);
        return 1;
    }

    printf("calloc: allocated 5 integers\n");

    // realloc
    a = realloc(a, 10 * sizeof(int));
```

```
// realloc
a = realloc(a, 10 * sizeof(int));

if (a == NULL)
{
    printf("realloc failed\n");
    free(b);
    return 1;
}

printf("realloc: resized a to 10 integers\n");

// free
free(a);
free(b);

printf("free: memory released\n");

return 0;
}
```

Power & Battery

```

arikel@DESKTOP-AMUSJME:~$ mkdir memory_allocation
arikel@DESKTOP-AMUSJME:~$ cd memory_allocation
arikel@DESKTOP-AMUSJME:~/memory_allocation$ pwd
/home/arikel/memory_allocation
arikel@DESKTOP-AMUSJME:~/memory_allocation$ nano memory.c
arikel@DESKTOP-AMUSJME:~/memory_allocation$ gcc -g memory.c -o
memory
arikel@DESKTOP-AMUSJME:~/memory_allocation$ ./memory
malloc: allocated 5 integers
calloc: allocated 5 integers
realloc: resized a to 10 integers
free: memory released
arikel@DESKTOP-AMUSJME:~/memory_allocation$ valgrind --version
valgrind-3.26.0
arikel@DESKTOP-AMUSJME:~/memory_allocation$ valgrind --leak-che
ck=full ./memory
==2839== Memcheck, a memory error detector
==2839== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Sewar
d et al.
==2839== Using Valgrind-3.26.0 and LibVEX; rerun with -h for cop
yright info
==2839== Command: ./memory
==2839==
malloc: allocated 5 integers
calloc: allocated 5 integers
realloc: resized a to 10 integers
free: memory released
==2839==
==2839== HEAP SUMMARY:
==2839==      in use at exit: 0 bytes in 0 blocks

```

```

==2839== HEAP SUMMARY:
==2839==      in use at exit: 0 bytes in 0 blocks
==2839==    total heap usage: 4 allocs, 4 frees, 1,104 bytes allo
cated
==2839==
==2839== All heap blocks were freed -- no leaks are possible
==2839==
==2839== For lists of detected and suppressed errors, rerun with
: -s
==2839== ERROR SUMMARY: 0 errors from 0 contexts (suppressed: 0
from 0)

```

```
printf("realloc: resized a to 10 integers\n");

// free
free(a);
//free(b);

printf("free: memory released\n");

return 0;
}
```

^G Help
^X Exit

^O Write Out
^R Read File

^F Where Is
^ Replace

^K Cut
^U Paste

^T Execute
^J Justify

```
memory
arikel@DESKTOP-AMUSJME:~/memory_allocation$ valgrind --leak-check=full ./memory
==2894== Memcheck, a memory error detector
==2894== Copyright (C) 2002-2024, and GNU GPL'd, by Julian Seward et al.
==2894== Using Valgrind-3.26.0 and LibVEX; rerun with -h for copyright info
==2894== Command: ./memory
==2894==
malloc: allocated 5 integers
calloc: allocated 5 integers
realloc: resized a to 10 integers
free: memory released
==2894==
==2894== HEAP SUMMARY:
==2894==   in use at exit: 20 bytes in 1 blocks
==2894==   total heap usage: 4 allocs, 3 frees, 1,104 bytes allocated
==2894==
==2894== 20 bytes in 1 blocks are definitely lost in loss record 1 of 1
==2894==    at 0x4857E43: calloc (vg_replace_malloc.c:1678)
==2894==    by 0x400124F: main (memory.c:21)
==2894==
==2894== LEAK SUMMARY:
==2894==   definitely lost: 20 bytes in 1 blocks
==2894==   indirectly lost: 0 bytes in 0 blocks
==2894==   possibly lost: 0 bytes in 0 blocks
==2894==   still reachable: 0 bytes in 0 blocks
==2894==   suppressed: 0 bytes in 0 blocks
==2894==
==2894== For lists of detected and suppressed errors, rerun with: -s
==2894== ERROR SUMMARY: 1 errors from 1 contexts (suppressed: 0 from 0)
arikel@DESKTOP-AMUSJME:~/memory_allocation$ |
```

Task 2: Develop a program that demonstrates Copy-on-Write behavior after a `fork()`. Analyze memory usage before and after modifying data in the child process.

```
arikela@DESKTOP-AMUSJME: × + ▾ - □ ×
GNU nano 8.7.1 cow.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

#define SIZE 1000000

int main()
{
    int *data = malloc(SIZE * sizeof(int));

    if (data == NULL)
    {
        perror("malloc");
        return 1;
    }

    for (int i = 0; i < SIZE; i++)
        data[i] = i;

    printf("Before fork:\n");
    printf("data[0] = %d\n", data[0]);

    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        free(data);
        return 1;
    }

    if (pid == 0)
```

^G Help	^O Write Out	^F Where Is	^K Cut	^T Execute
^X Exit	^R Read File	^\ Replace	^U Paste	^J Justify

```

if (pid == 0)
{
    // Child process
    printf("\nChild before modification:\n");
    printf("data[0] = %d\n", data[0]);

    data[0] = 9999;

    printf("Child after modification:\n");
    printf("data[0] = %d\n", data[0]);

    sleep(2);
    free(data);
    exit(0);
}
else
{
    // Parent process
    sleep(1);

    printf("\nParent after child modification:\n");
    printf("data[0] = %d\n", data[0]);

    wait(NULL);

    free(data);
}

return 0;
}

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify


```
arikel@DESKTOP-AMUSJME:~$ mkdir copy_on_write
arikel@DESKTOP-AMUSJME:~$ cd copy_on_write
arikel@DESKTOP-AMUSJME:~/copy_on_write$ nano cow.c
arikel@DESKTOP-AMUSJME:~/copy_on_write$ gcc cow.c -o cow
arikel@DESKTOP-AMUSJME:~/copy_on_write$ ./cow
Before fork:
data[0] = 0

Child before modification:
data[0] = 0
Child after modification:
data[0] = 9999

Parent after child modification:
data[0] = 0
arikel@DESKTOP-AMUSJME:~/copy_on_write$ |
```